

表 5.2 搬运量与完成时间权衡反例分析（C04）

用例名称	本文主算法 / cycles	C04 对照方案 / cycles	相对变慢幅度
case_005	33,515	38,209	+14.01%
case_016	2,091,375	2,399,125	+14.72%
case_019	16,247	28,514	+75.50%
case_025	942,938	975,658	+3.47%
case_039	449,653	494,942	+10.07%
case_075	343,930	452,120	+31.46%

尤其在 case_019、case_039 与 case_075 中，C04 方案的边界 COPY 字节数低于本文主算法，但其执行完成时间却出现了大幅退化（在 case_019 中变慢达 75.50%）。

图 5.4-4 与图 5.4-5 分别给出了 case_019（5 核心场景）在官方离散事件仿真下的全景与局部时序对比。数据显示，C04 方案的边界 COPY 字节数低于主算法（额外搬运量减少 73.20%），且无片上容量不足引发的 Step 2 缓存换入换出（spill 为 0），但其最终执行完成时间增加了 75.50%（Makespan 由 16,247 cycles 增至 28,514 cycles）。图中各核心横道分别对应各个核心的 Task 与 Pipe 活跃区间（例如 C0:T0 与 C3:T3 分别表示 Core 0 与 Core 3 上排定的具体 Task 任务段）。在全景时序中，主算法于 16,247 cycles 即完成了全图计算，而 C04 方案持续运行至 28,514 cycles。在 13,800 至 17,400 cycles 的局部放大窗口中，C04 方案在各个核心的 Task 之间存在未记录流水线操作的间隔。需要严格说明的是，图中未着色的空白区域仅代表仿真器在此时间段内未调度对应的流水线事件，客观反映对应流水线未记录操作占用，此处不作未经微架构事件级归因分析的机理臆测。

上述限定反例表明：单纯优化静态通信数据量并不能直接保证任务执行时间的缩短。

5.5 本章小结

本章针对同核心任务合并执行与片上私有缓存跨子图复用语义，构建二核心局部超图割模型以最小化跨核数据复制代价，并引入计算流水线工作量上限守卫（Load Guard）抑制偏载。在 5 核心场景下平均加速比达到 4.5498，相比前一完整版本在 500 个测试点中 268 个缩短 Makespan 且零退化。超图割仅优化换入换出前的数据复制量，工作量上限不保证片上私有缓存无溢出（spill）。

6 问题三：共享只读缓存下的多核调度

6.1 问题分析与缓存访问约束

问题三在问题二的多核心硬件任务（Task）合并执行模型基础上，引入了全部计算核心共享的片上只读缓存（Shared Read-only Cache，后文简称为 Cache）。该 Cache 容量固定为 1 MiB（1,048,576 字节），只读缓存命中读取总带宽为 250 B/cycle。在这一设定下，各个计算核心除了访问各自片上私有的 L1 Buffer 与 Unified Buffer（UB）之外，符合条件的 COPY_IN 搬入操作可优先向该共享 Cache 发起读取查询，以减少访问外部主存（DDR，所有并发 DDR 传输共享总带宽为 60 B/cycle）带来的时钟周期开销；若发生缓存未命中，则在对应数据经由 DDR 链路彻底搬运完成的时刻，方被正式写入并插入 Cache。

依据官方评测程序的具体实现，共享只读 Cache 的访问行为与状态迁移严格遵循如下规则：

1. **查询时机与命中判定：**当某个核心启动数据搬入操作 COPY_IN、能够确定待搬入张量的全局逻辑标识且数据大小大于 0 字节时，该操作在启动时刻立即向 Cache 发起查询。若

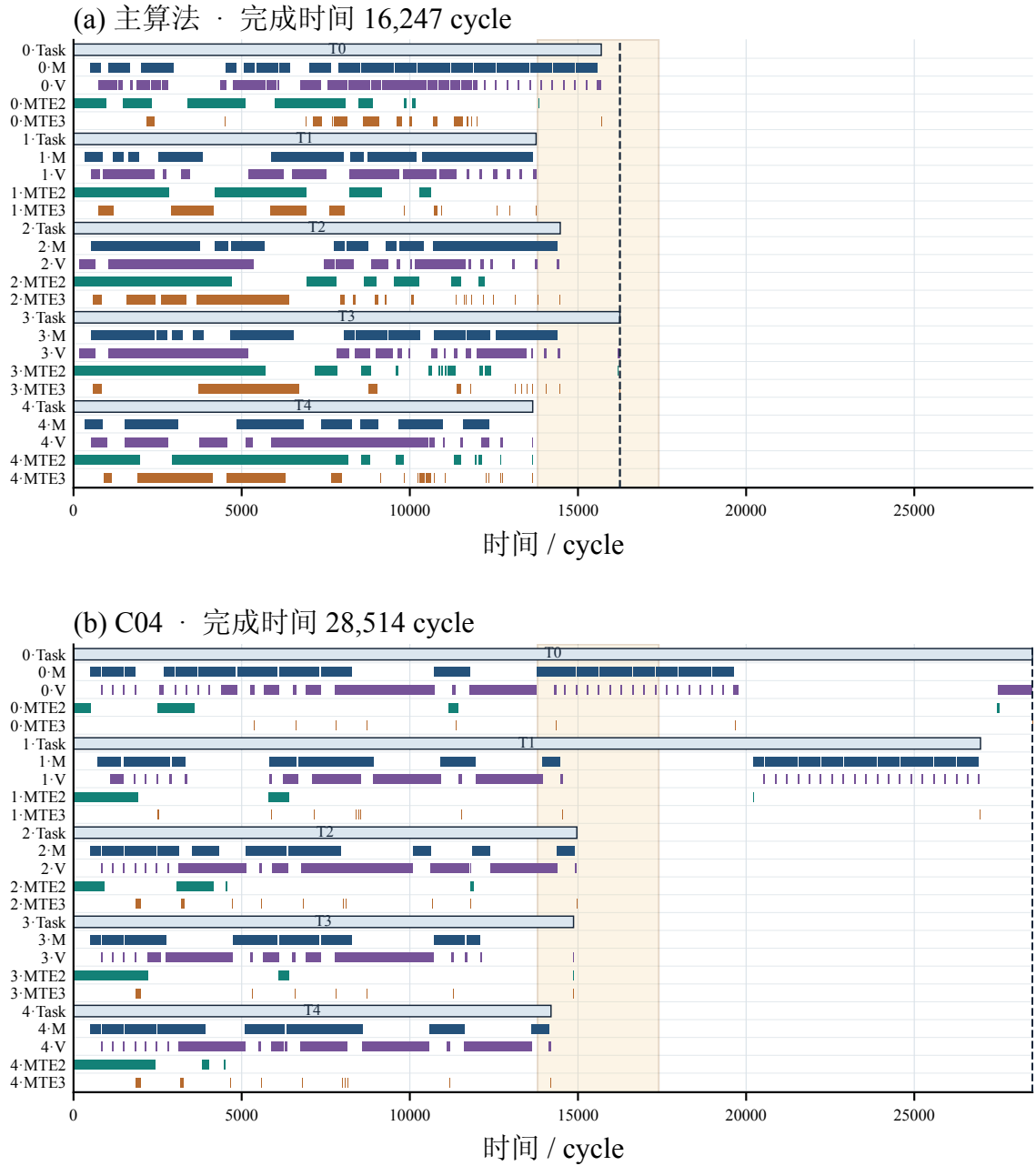


图 5.4-4 case_019 (5 核心) 主算法与 C04 对照方案全景执行时序对比。展示两方案在全流程时间尺度 (0 至 28,514 cycles) 下各核心 Task 与 Pipe 活跃区间；C04 方案无容量不足引起的缓存换入换出 (spill 为 0)，但完成时间 (28,514 cycles) 劣于主算法 (16,247 cycles)。

Cache 中已存在该逻辑标识对应的数据块，则判定为**缓存命中** (cache hit)，数据直接经由 Cache 读入该核心的片上私有缓存，数据传输速率受 Cache 带宽限制；若 Cache 中不存在该数据，则判定为**缓存未命中** (cache miss)，核心必须向片外 DDR 发起读取请求。

2. **数据插入与在途约束**：当发生缓存未命中时，数据通过 DDR 链路进行搬运，并受系统共享总带宽 (60 B/cycle) 及并发链路仲裁约束。**数据仅在 DDR 搬运彻底完成的时刻，方被正式写入并插入 Cache。**若某个张量正在由其他核心从 DDR 读取中 (传输尚未结束)，后继核心发起的相同张量读取请求由于 Cache 中尚无完整有效副本，无法被判定为命中，依然作为未命中处理；

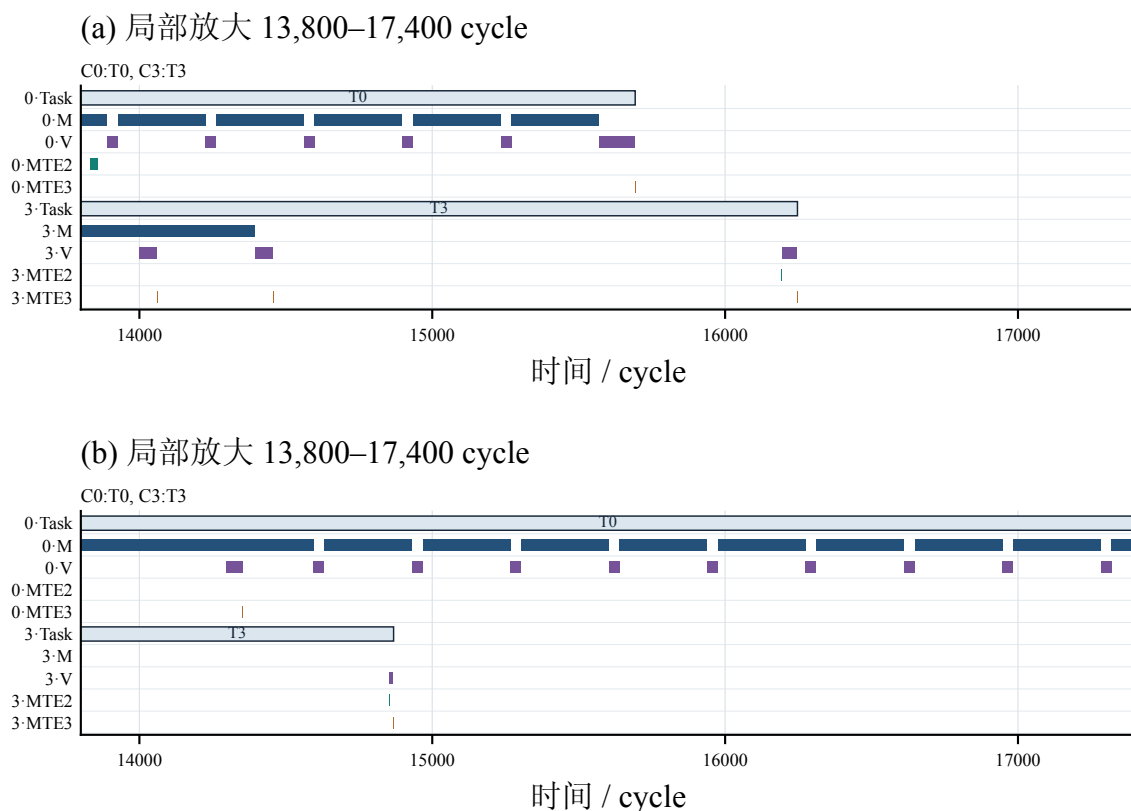


图 5.4-5 case_019 (5 核心) 两方案在关键局部时间窗 (13,800 至 17,400 cycles) 的时序放大对比。展示主算法与 C04 在 Core 0 (C0:T0) 及 Core 3 (C3:T3) 等任务上的流水线事件分布。图中未着色空白区域仅客观反映对应流水线未记录操作占用, 不作未经事件级归因分析的机理臆测。

3. **先入先出替换策略 (FIFO):** 当 Cache 当前占用总字节数与待插入张量大小之和超过容量上限 (1 MiB) 时, Cache 依据各数据块的首次插入时间先后顺序 (通过有序队列维护并按占用总字节数进行容量控制), 依次逐出最早存入的数据块, 直至释放出足够的总字节容量以容纳新张量。该共享只读缓存仅对总字节数进行总量控制, 不涉及物理地址连续性或碎片约束, 并非核心私有的 L1/UB 物理分配器。**缓存命中操作不会更新或刷新该数据块的入队时间戳, 亦不改变其在 FIFO 队列中的淘汰顺序;**
4. **超大张量不插入规则:** 对于单份数据大小严格超过 Cache 总容量 (1 MiB) 的张量, 系统直接通过 DDR 链路完成搬运, 绝不尝试将其写入 Cache, 以避免无意义地清空 Cache 中已有的其他活跃数据;
5. **硬件同步与数据依赖不变性:** 共享 Cache 的引入并未免除计算图原有的前序数据依赖约束, 亦未消除跨核心数据传递时必须满足的 500 cycles 硬件同步等待。即使后继核心从 Cache 中命中了前序算子生成的数据, 该读取操作依然必须在前序算子写出操作完成 500 cycles 之后方可启动。

上述时序规则表明, 调度算法排定的算子执行顺序直接决定了各张量读取请求在时间轴上的先后位置。由于 Cache 具有容量上限与 FIFO 逐出机制, 单纯统计静态命中次数或总命中字节数, 并不能直接反映程序完成时间 (Makespan) 的变化趋势。

根据 case_021 在 3 核心配置下的官方离散事件仿真记录, 张量 t1000000001 (数据大小为 4096 B) 的生命周期 (如表 6.1 所示) 呈现了典型的 Cache 状态迁移过程。由于

FIFO 队列仅依据首次插入时间排序，第 4956 cycle 的命中读取并未刷新其在队列中的时间戳，该张量最终于第 323850 cycle 因新张量插入导致容量超限而被逐出。

表 6.1 同一张量的 Cache 事件

时刻 / cycle	操作核心与动作	缓存事件类型	状态与队列行为说明
0	核心 0 发起 COPY_IN	缓存未命中 (Miss)	向片外 DDR 发起读取传输
207	核心 0 完成 DDR 搬运	插入缓存 (Insert)	写入 Cache 并进入 FIFO 队列尾部
4956	核心 2 发起 COPY_IN	缓存命中 (Hit)	直接经 Cache 读取，不改变 FIFO 队列次序
323850	核心 2 插入 t1000000487	缓存逐出 (Evict)	插入 6144 B 数据导致容量超限，最早存入者被逐出

更进一步，case_021 (3 核心场景) 在同一调度方案 (SHA 13362774...) 下的成对对比呈现了一个关键的非单调反例：方案在两配置下的 8,881 条操作按 (core_id, task_id, op_id) 完全对齐，静态类型与各类别搬运字节数完全一致 (调度复制总量 39,993,344 字节，额外 DDR 搬运量 36,655,104 字节，溢出换出量 31,944,704 字节)；但在开启只读 Cache 后，全局完成时间由无 L2 运行的 2,140,720 cycles 增至 2,140,863 cycles (变慢 143 cycles，相对增益 CacheGain $\approx 0.999933 < 1$)。

图 6.1-1 与图 6.1-2 展示了官方仿真器推进下的微架构事件时间线：

1. **全景时间线对比 (图 6.1-1)**：展示了三核心在四条执行流水线 (MTE2 搬入、M 矩阵乘、V 向量计算、MTE3 搬出) 上的全量推进过程，两方案的事件行序与持续时长均按官方仿真实际记录绘制；
2. **命中提前与总线争用局部窗口 (图 6.1-2)**：
 - 子图 (a) 捕捉到了首次 Cache 命中行为 (Core 2, 4,900 至 7,000 cycles 窗口)：算子 1000004905 成功命中 Cache，读取耗时由 207 cycles 骤降为 17 cycles，触发后继算子 1000006255 提前至 4,973 cycles 启动；
 - 子图 (b) 记录了后续 DDR 读取变慢现象 (Core 0, 443,050 至 443,250 cycles 窗口)：算子 1000006089 在两配置下均于 443,077 cycles 同时发起 DDR 读取，但开启 Cache 后的搬运耗时由 69 cycles 延长至 138 cycles (全表同起点变慢的搬入操作共 3 笔)。
3. **因果边界与非单调性结论**：需要客观指出的是，上述局部窗口仅展示了事件级别的对比观测。虽然同起点 DDR 读取变慢与缓存引入导致的全局事件交错及总线争用机制相容，但在全流程 8,881 条操作中共有 7,485 条操作的启动时刻发生了微小偏移，因此不能武断宣称单一局部事件独自因果解释了末端增加的全部 143 个周期。这一反例充分证明了在共享总线与离散事件仲裁环境下，缓存命中并不自动保证整体完成时间单调下降。

6.2 基于归约树的存储峰值模型

6.2.1 归约树表示与存储峰值

在多核心调度过程中，除了跨核心通信与只读缓存优化外，各个计算核心内部的算子执行次序对片上私有存储的驻留峰值具有决定性影响。在任务树内存受限调度领域，

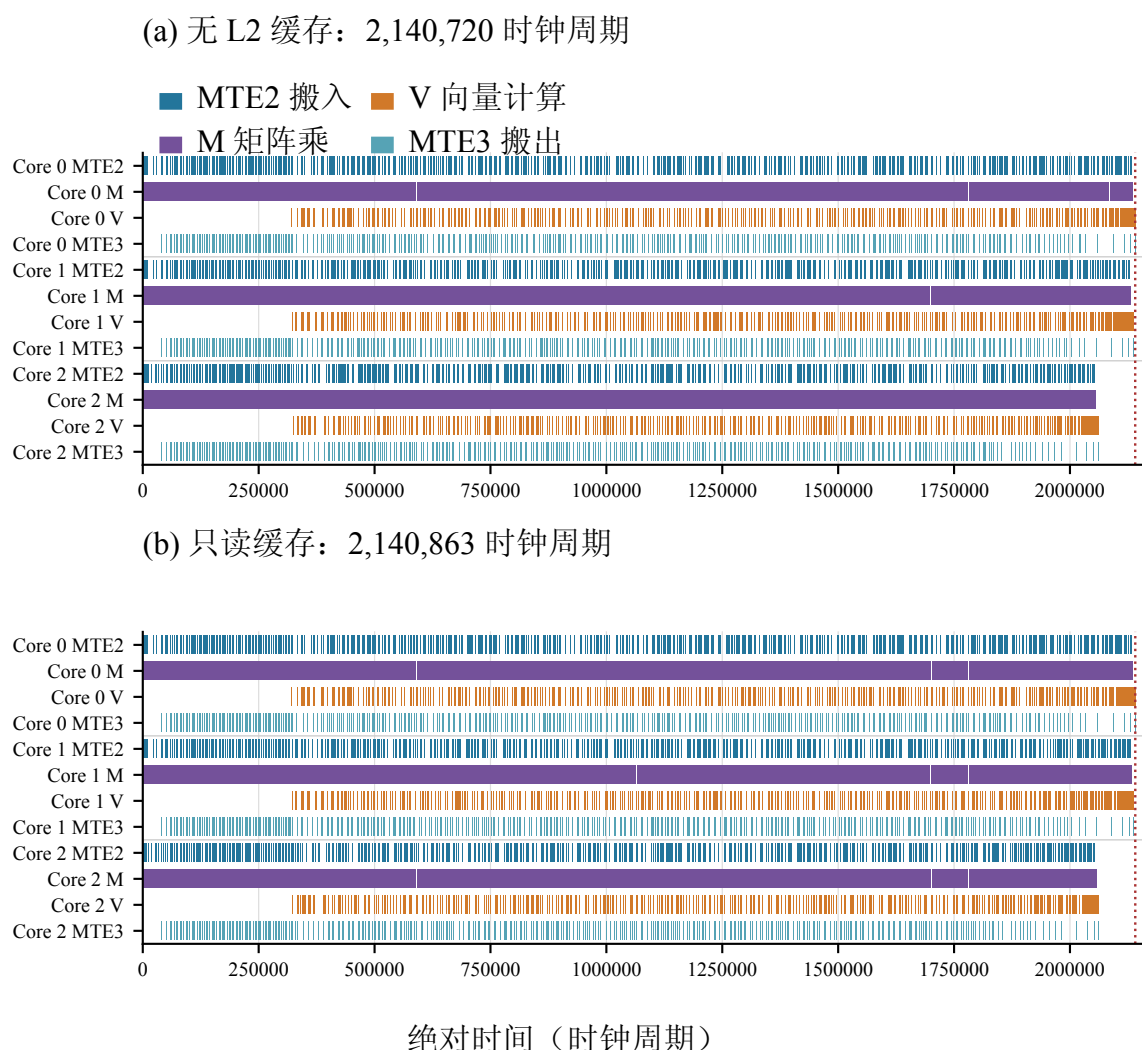


图 6.1-1 case_021（3 核心）同一方案开启 Cache 前后的全景时序对比。展示无 L2 运行（上图，完成时间 2,140,720 cycles）与开启只读 Cache（下图，完成时间 2,140,863 cycles，变慢 143 cycles）在 Core 0、Core 1、Core 2 及四类流水线上的真实事件时间线；方案结构与搬运字节完全相同。

Eyraud-Dubois 等 [17] 深入探讨了父算子输出预先分配、子算子输出暂留以及存储峰值与执行时间之间的权衡关系。需要明确的是，文献 [17] 考虑了任务执行所需的程序文件存储占用（execution files），而本文树状模型仅统计内部生成的张量数据，未计入此类任务执行额外开销；同时 NPU 具有片上独立的双私有存储池与 FIFO 缓存逐出规则，不能直接平移文献 [17] 中的理论近似界或最优性结论。

考虑由同一父算子汇聚的两个互不依赖的计算分支（子树）A 与 B。父算子必须等待 A 与 B 的全部计算操作执行完毕，方可读取两者的输出张量以执行归约计算。若算法选择先执行分支 A，则分支 A 计算完成后生成的输出张量不能立即释放，必须持续驻留在片上存储中，直至分支 B 完整执行完毕且父算子完成读取。此时，分支 A 的输出数据将与分支 B 在计算过程中产生的各类活跃中间张量产生空间共存。反之，若算法选择先执行分支 B，片上存储在各个时刻所保留的数据量将发生改变。

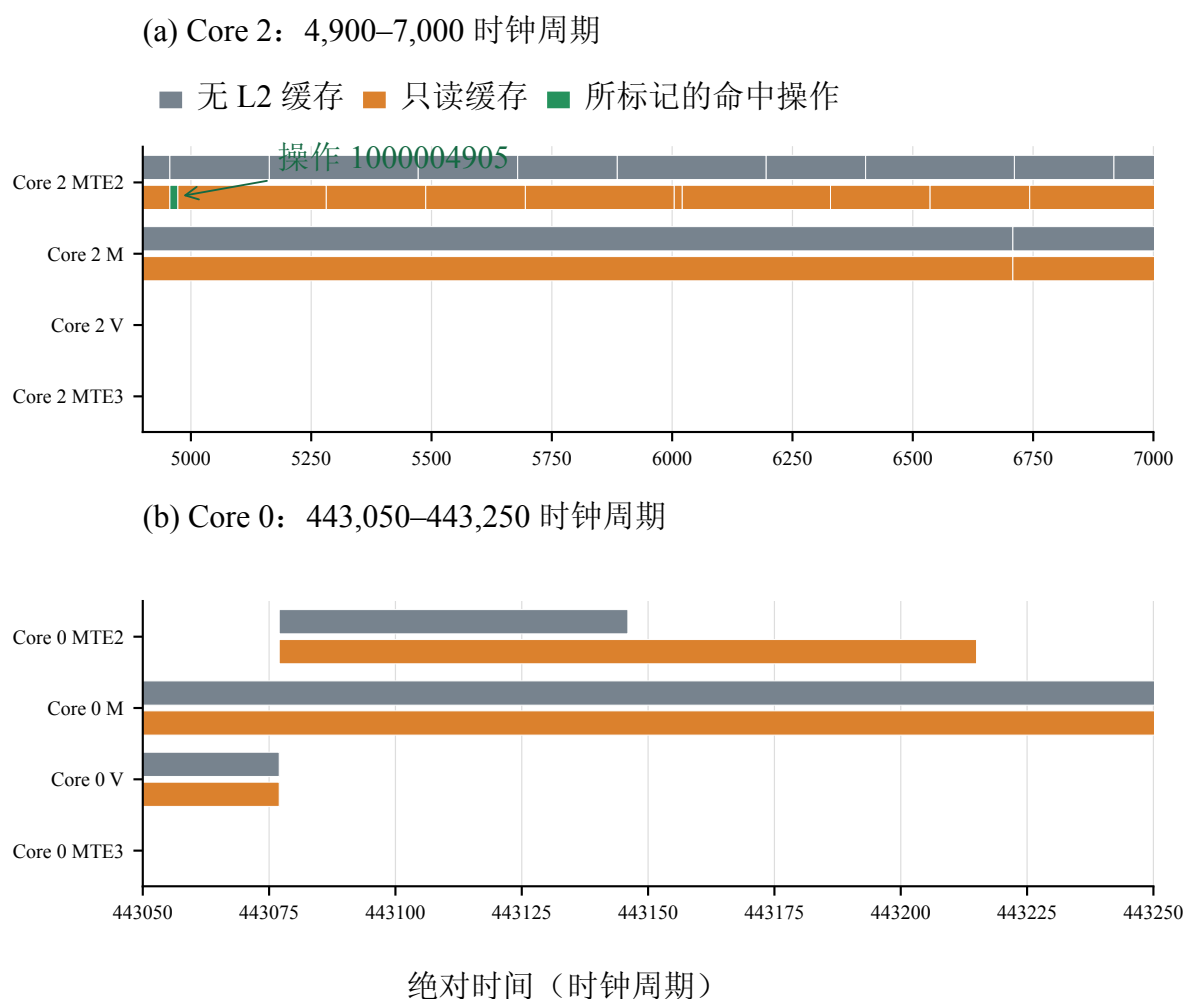


图 6.1-2 case_021（3 核心）同一方案开启 Cache 前后的关键局部事件窗口时序放大。（a）窗口 1（Core 2, 4,900–7,000 cycles）：算子 1000004905 因 Cache 命中耗时自 207 cycles 降至 17 cycles（绿色高亮），使后序算子 1000006255 提前启动；（b）窗口 2（Core 0, 443,050–443,250 cycles）：算子 1000006089 在两配置下同在 443,077 cycles 启动，DDR 访问耗时由 69 cycles 延长至 138 cycles。

为了量化不同子树执行次序对存储占用的影响，我们首先定义一类基础的**树状计算结构模型**：

1. 结构仅包含单棵归约树内部的算子，每个算子仅生成一个确定大小的输出张量；
2. 每个内部算子的输出张量仅被其唯一的父算子读取，不被外部其他分支复用；
3. 子树之间采用非交错（non-interleaved）方式执行，即完整处理完一棵子树的全部算子后，方开始执行下一棵子树；
4. 存储分配语义满足：父算子的输出张量存储空间必须在子算子的输出张量被完全释放之前分配。

对于子树 i ，定义 h_i 为完整执行该子树期间内部计算结果所占用的最大存储总字节数（存储峰值），定义 r_i 为该子树全部执行完成后必须持久保留的根节点输出总字节数。在此模型下，各数值满足物理约束 $h_i \geq r_i \geq 0$ 。模型仅统计该子树内部生成并保留的数据张量，不包含全图外部输入，亦不细分 L1 Buffer 与 UB 的独立物理边界。

以包含两个分支 A、B 的数学设定为例进行说明：设分支 A 的参数为 $(h_A, r_A) = (8, 2)$ KiB，分支 B 的参数为 $(h_B, r_B) = (7, 5)$ KiB，父算子自身的输出大小为 $r_{\text{parent}} = 3$ KiB。若按“先 A 后 B”的顺序执行：

1. 执行分支 A 期间，内部存储峰值为 8 KiB；
2. 分支 A 执行完成，保留输出 2 KiB；随后执行分支 B，由于分支 B 自身内部峰值为 7 KiB，此时片上存储峰值为 $2 + 7 = 9$ KiB；
3. 分支 B 执行完成，保留输出 5 KiB，与分支 A 保留的 2 KiB 合计占用 7 KiB。在父算子启动计算时，先分配父算子输出 3 KiB，此时总存储占用达到 $2 + 5 + 3 = 10$ KiB；随后父算子读取并释放 A 与 B 的输入数据。

整个流程中的最大存储占用峰值为 $\max(8, 9, 10) = 10$ KiB。

反之，若按“先 B 后 A”的顺序执行：

1. 执行分支 B 期间，内部存储峰值为 7 KiB；
2. 分支 B 执行完成，保留输出 5 KiB；随后执行分支 A，片上存储峰值上升至 $5 + 8 = 13$ KiB；
3. 分支 A 执行完成，两分支输出合计 7 KiB，分配父算子输出时达到 $5 + 2 + 3 = 10$ KiB。

该顺序下的最大存储占用峰值为 $\max(7, 13, 10) = 13$ KiB。树状子树处理次序与理论存储峰值推导见图 6.2-1。需要指出的是，该 10 KiB 与 13 KiB 属于前文定义的高层树状模型理论推导，而非底层微架构硬件实测峰值。对比可见，调整非交错子树的执行先后次序，使理论存储峰值在 10 KiB 与 13 KiB 之间产生了明确差异。

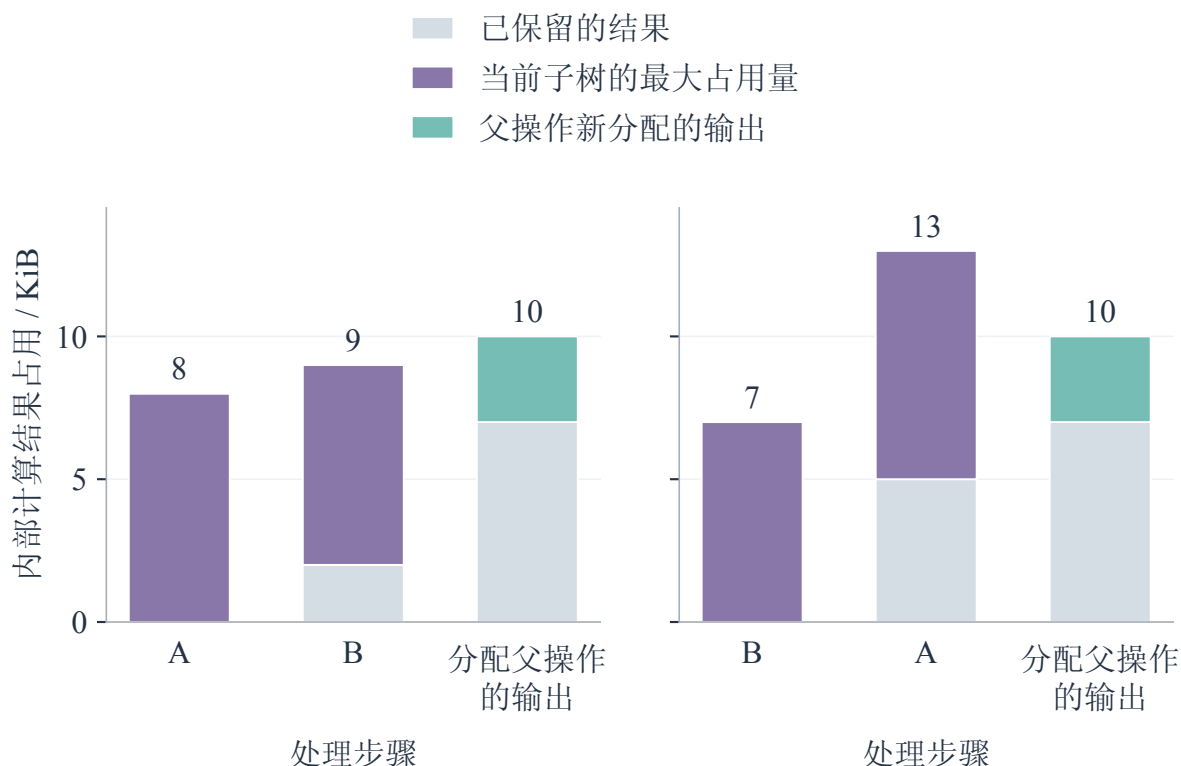


图 6.2-1 两种子树处理顺序的内部结果占用。左：先 A 后 B，峰值 10 KiB；右：先 B 后 A，峰值 13 KiB。横轴为处理步骤，不是模拟时钟；数字为第 6 章的公式示例。

算法 6.1 归约森林核心排序与 DFS 局部步骤

```

1 # 输入: 拓扑索引 index、核心归属 assignment、树属性 records 与张量尺寸
   tensors
2 # 输出: 各核心的算子执行序列 sequences (单操作子图约束)
3 peak, retained, child_order = {}, {}, {}
4 # 沿拓扑序自底向上递推: 子树按 (peak - retained) 降序排序, 相同按操作 ID
   升序破平
5 for u in index.order:
6     children = records[component_for[u]]["children"][u]
7     children.sort(key=lambda c: (-(peak[c] - retained[c]), c))
8     child_order[u] = children
9     live, frontier = 0, 0
10    for c in children:
11        frontier = max(frontier, live + peak[c])
12        live += retained[c]
13    own_output = tensors[output_tensors[u][0]]
14    retained[u] = own_output
15    peak[u] = max(frontier, live + own_output) # 父输出在子树输出释
        放前分配
16
17 # 依排定子节点次序进行后序遍历 (伪代码抽象: DFS 后序遍历生成各树执行序
   列)
18 postorders = {min(r["members"]): dfs_postorder(r["root"],
   child_order)}
19         for r in records}
20 # 各核心对所分配分量按最小算子 ID 排序并拼接执行序列
21 sequences = []
22 for component_ids in assignment:
23     ordered = sorted(component_ids, key=lambda cid: min(index.
   components[cid]))
24     sequences.append([u for cid in ordered
25                       for u in postorders[min(index.components[cid])
   ]])
26 return sequences

```

源码对应: 311322b, src/q3/forest_memory_order.py::construct (第 153–192 行; 其中 `dfs_postorder` 为伪代码抽象, 封装了按排定子节点次序的显式栈 DFS 后序遍历)。算法在非交错树模型下按子树 $h_i - r_i$ 降序排定处理次序, 递推循环中 `live` 维护已处理子树输出总和、`frontier` 维护峰值包络, 并在子树输出释放前计入父算子输出 `own_output`; 外围包含结构守卫与单操作子图映射。

6.2.2 非交错子树的最优排序及证明

设某个父算子拥有 m 棵待处理的子树, 记 $\pi = (\pi_1, \pi_2, \dots, \pi_m)$ 为这 m 棵子树的一个非交错处理排列次序, 父算子自身输出张量大小记为 r_{parent} 。

在排列 π 下, 当算法开始处理第 j 棵子树 π_j 时, 前面已经处理完毕的 $j - 1$ 棵子树所保留的根节点输出已累计占用 $\sum_{\ell < j} r_{\pi_\ell}$ 字节, 而处理当前子树 π_j 额外需要的最大内部临

时存储为 h_{π_j} 字节；当全部 m 棵子树均处理完毕、父算子开始分配自身输出张量时，所有子树的输出尚未被释放，此时存储占用为 $\sum_{i=1}^m r_i + r_{\text{parent}}$ 字节。因此，排列 π 对应的全局存储峰值函数 $H(\pi)$ 精确表示为：

$$H(\pi) = \max \left\{ \max_{1 \leq j \leq m} \left(\sum_{\ell < j} r_{\pi_\ell} + h_{\pi_j} \right), \sum_{i=1}^m r_i + r_{\text{parent}} \right\}. \quad (6.1)$$

该式中各项的物理单位均为字节（B）。

命题 6.1（子树存储最优排序）。若对所有子树均有 $h_i \geq r_i \geq 0$ ，则在上述非交错执行且父输出先分配的模型中，将子树按照指标差值 $h_i - r_i$ 从大到小降序排列，能够使全局存储峰值 $H(\pi)$ 取得最小值。

证明. 考虑排列中任意两个相邻的子树 A 与 B，设在当前序列中 A 紧邻且位于 B 之前。假设两者的指标满足 $h_A - r_A \geq h_B - r_B$ 。

在计算峰值时，暂时不考虑在处理 A 之前已经保留在存储中的其他前序子树输出常量 $R_{\text{prev}} = \sum_{\ell < \text{pos}(A)} r_{\pi_\ell}$ 。若按“先 A 后 B”的相对顺序执行，这两步操作期间局部达到的最大存储占用为：

$$H_{AB} = \max(h_A, r_A + h_B). \quad (6.2)$$

若将 A 与 B 的执行顺序对调为“先 B 后 A”，对应的局部最大存储占用为：

$$H_{BA} = \max(h_B, r_B + h_A). \quad (6.3)$$

由假设条件 $h_A - r_A \geq h_B - r_B$ 移项可得：

$$r_A + h_B \leq r_B + h_A. \quad (6.4)$$

同时，由于物理保留量非负（ $r_B \geq 0$ ）且子树内部峰值不小于保留量（ $h_A \geq r_A \geq 0$ ），显然有：

$$h_A \leq r_B + h_A. \quad (6.5)$$

综合上述两式，可得：

$$H_{AB} = \max(h_A, r_A + h_B) \leq r_B + h_A \leq \max(h_B, r_B + h_A) = H_{BA}. \quad (6.6)$$

前序已保留的常量 R_{prev} 对两种次序的每一项产生完全相同的线性位移，不改变不等关系。并且，当 A 与 B 均处理完毕后，两者留在存储中的数据总和恒为 $r_A + r_B$ ，对后续其他子树的处理过程以及父算子输出 r_{parent} 的分配不产生任何状态影响。

因此，若某个排列中存在相邻子树不满足 $h_i - r_i$ 降序，将其交换位置绝不会导致全局存储峰值 $H(\pi)$ 增大。通过有限次相邻逆序对的对调操作，任意排列均可转化为按 $h_i - r_i$ 单调递减排列，且其峰值不减。自底向上递推应用于整棵树的各个归约节点，即证该贪心排列能够使模型内的存储峰值达到最优。□

该证明借鉴了经典树表达式代码生成与存储优化思想 [18]，并在本文所述的限定模型下给出了按字节度量存储占用的相邻交换证明。

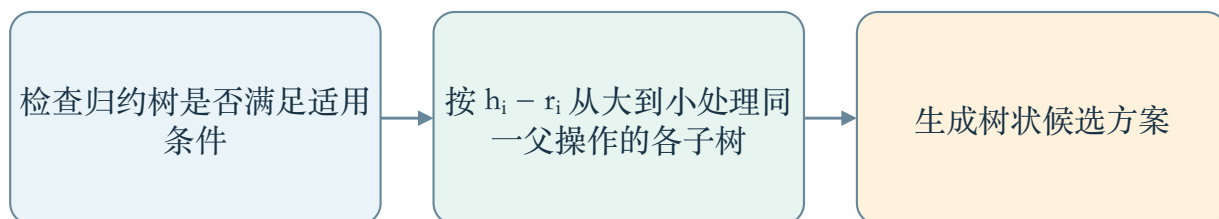
需要强调的是，命题 6.1 证明的最优性受到严格的**模型适用边界**约束：

1. 结论仅适用于单棵孤立归约树、内部纯张量数据依赖、子树非交错处理的理想化模型；

2. 该模型未考虑多核之间的并行交错、外部输入张量的重叠生命周期、L1 Buffer 与 UB 的独立物理容量分割；
3. 该模型所得的标量存储峰值是用于指导排序的启发式代理（ordering proxy），它并不等价于官方底层编译流程中绝对不发生 Step 2 缓存换入换出（spill），亦不直接等价于离散事件仿真器推进下的多核 Makespan 最优。近期关于峰值内存调度的理论研究表明，面向一般有向无环图寻求支配调度或保持局部最优的拓扑线性化，需要特定的结构隔离条件 [19]；本文针对非交错归约树给出的 $h_i - r_i$ 交换证明受限于单树模型，不外推为多流水线、动态 FIFO Cache 与完整 Makespan 的全局最优解。

6.3 基于归约树排序的模型求解

(a) 归约树中各子树的处理顺序



(b) 树状方案的评价与选择



图 6.3-1 树状分支排序准则与完整候选方案的评测决策流程。展示基于指标差值 $h_i - r_i$ 降序排定非交错子树次序以最小化树模型局部峰值存储，以及共享至多 3 次在线 E0 评测预算下的前置检查、与当前最优方案直接比对编码去重、下界剪枝与完成时间改善选优决策链。存储优化属于非交错树模型推导，不改变分核归属，非物理片上缓存实测峰值。

6.3.1 归约树识别与执行序列构造

在算法实现中，为了将上述单树排序理论应用于输入计算图，程序构建了带有结构守卫的森林内存排序模块（`forest_memory_order.py`）。该模块并不假定所有输入图均能匹配树状模型，而是在进入优化前执行严密的结构验证。

归约树森林的结构判定 在本算法实现中，森林内存排序模块（`forest_memory_order.py`）针对输入计算图设定了明确的工程接受条件（`guard conditions`）。这些条件界定了当前代码所支持的结构范围，并非归约森林在数学图论上的充要定义：

1. **多连通分量要求**：计算图在收缩边界 COPY 节点后的计算依赖图中，其无向连通分量数量必须至少为 2（`len(index.components) >= 2`）；
2. **各分量单一归约根**：每个连通分量内部必须构成一棵有向树，且每个分量包含且仅包含一个没有计算后继算子的根算子（`root op`）；
3. **汇合节点存在性**：整张计算图中必须至少存在一个拥有 2 个及以上前序输入算子的汇聚算子（`join op, joined_nodes >= 1`）；
4. **计算出度无分叉**：除根节点外部输出外，所有计算算子的计算后继算子数量至多为 1（无 `fanout, outdegree <= 1`）；
5. **张量属性与唯一性**：计算图中所有张量具备已知的非负整数尺寸，且每个计算算子恰好生成一个内部输出张量；
6. **边界搬运严格规范**：计算图仅包含合法的边界 COPY_IN 与 COPY_OUT 算子，其中 COPY_IN 必须将 DDR 外部输入映射为 L1 片上输入，COPY_OUT 必须将根算子的 L1 输出写回 DDR，且输入输出张量大小完全一致；
7. **不支持直接算子依赖边**：若计算图中包含直接算子依赖边（`direct op-op edges`），由于无法确定对应的数据字节大小，模块将其视为超出当前排序实现范围的不支持结构，主动抛出异常（`UnsupportedStructure`）。

上述结构守卫明确了本实现的工程支持边界；满足上述条件的输入图仍须经受实际评测的检验，算法不宣称满足这些条件即与全部底层硬件容量特征严密对应。

核心分配与后序遍历 在通过上述结构检验后，算法并不打破或重新搜索算子的计算核心分配，而是严格保留由基准索引确定的核心归属映射（`index.assignment(cores)`）。

算法以各连通分量为独立单元，自底向上递归计算各算子的 (h_u, r_u) 属性，在每个汇聚节点处将前序子节点按 $h_i - r_i$ 降序排序（相同指标下按算子 ID 破平）。随后，通过深度优先后序遍历（DFS postorder）为每个树分量生成算子执行序列。最后，在每个计算核心内部，将指派给该核心的各分量序列按照分量最小算子 ID 依次拼接，构造出满足单操作子图约束的候选调度方案。

6.3.2 分层候选方案构造

在问题三的算法体系中，顶层求解器 `forest_solve` 嵌套调用一系列前序求解模块，其调用链关系为：

$$\text{calendar_solve} \longrightarrow \text{pipeline_solve} \longrightarrow \text{witness_solve} \longrightarrow \text{forest_solve}. \quad (6.7)$$

各求解模块在执行过程中依次尝试不同构型的候选方案，并严格共用全局至多 3 次的官方在线 E0 评测预算。

候选方案的生成次序

1. **calendar_solve**: 在问题二空闲区间调度的基础上，引入单核单操作子图的保底锚点 (anchor)，并针对大模型注意力计算 (attention route) 尝试构造基于 gap 的核心放置方案；
2. **pipeline_solve**: 调用 calendar_solve 获得基准方案后，针对具有共享输入特征的多核流水线结构，调用 shared_pipeline.construct 构造候选方案。该模块要求非空连通分量必须为相邻拓扑位置均存在直接依赖的有向链，且每条链均读取相同的外部输入张量；各链对应位置的流水线类型、计算时长以及共同输入读取签名完全一致；且链长（位置数）不超过 512、分量数不小于核心数 K 。算法提取代表链的计算时长序列，利用动态规划将其切分为 $\min(K, \text{位置数})$ 个连续段，最小化各段工作量和的最大瓶颈，进而按阶段构建核心执行序列。该连续极小化极大划分仅作为平衡各核心计算时长的代理目标，并不直接等同于官方最终的 Makespan；
3. **witness_solve**: 调用 pipeline_solve 获得前序最优方案后，若注意力计算路线在重新排序后评估代理变差，则生成一份保留原放置见证的候选方案 (attention witness)；
4. **forest_solve**: 作为顶层求解模块，首先完整调用 witness_solve 获取当前已知最优方案 (incumbent winner)。若满足归约森林结构守卫，则生成基于本章树排序理论的 forest_memory_order 候选方案。

评测预算与终止条件 整个求解流程在单次运行时严格共用至多 3 次官方 E0 在线评测预算 (total_e0_limit = 3)。在 forest_solve 阶段，系统对候选方案的状态转移实施精细化管理：

1. **预算耗尽跳过 (skip: three_call_budget_already_used)**: 若前序父求解器（如 calendar、pipeline、witness）已累计发起了 3 次评测请求，则当前阶段不再生成亦不再评测 forest 候选方案，直接输出当前最优方案；
2. **结构不支持跳过 (skip: UnsupportedStructure)**: 若输入计算图不满足归约森林的结构守卫，模块安全捕获异常并跳过，保持当前方案输出；
3. **方案完全重复 (duplicate)**: 若生成的 forest_memory_order 方案与当前最优方案的代码表示完全一致，则记录为重复状态，不发起重复评测；
4. **完成时间下界剪枝 (bound_pruned)**: 对生成的候选方案求解其不可避免的物理完成时间下界 L 。若 $L \geq U$ (U 为当前已知最优方案的官方 Makespan)，则该候选在数学上绝无可能超越当前最优解，算法直接放弃评测；
5. **下界不支持但允许评测 (UnsupportedBound)**: 若候选方案超出下界分析模块支持的范畴，系统仅记录 bound_unavailable，绝不直接拒绝候选方案，依然允许其进入后续评测流程；
6. **评测执行与异常处理**: 若通过全部前置检验，在剩余预算内调用官方 E0 进行评测。若评测抛出数据校验异常 (EvaluationValidationError)，模块捕获后记录 rejected 并安全保留当前最优解；若评测成功且最终 Makespan 严格小于当前最优解 ($M < U$)，方接受该方案并替换为新的最优解。在问题三的决策逻辑中，仅以 Makespan 严格改善为采纳标准，不使用 DDR 搬运量进行平局决选 (tie-break)。

6.3.3 固定候选方案的完成时间下界

为了在发起昂贵的官方评测之前安全剔除无希望的候选方案，算法设计了针对静态调度方案的完成时间下界分析器 (pipe_bound.py)。

必要时序不等式与下界计算 该分析器 (pipe_bound.py) 针对已完全排定次序的静态调度方案生效，其适用范围受如下守卫条件 (guards) 严格约束：

1. **算子与流水线范围**: 每个子图必须恰好包含一个原始非 COPY 算子, 且所有计算算子的流水线类型仅限 PIPE_M 与 PIPE_V;
2. **计算周期非负整数**: 算子计算时长满足 $d_u = \max(1, \text{cycles}_u)$, 其中原始 cycles_u 必须为非负整数;
3. **依赖提取与保守拒绝**: 依赖关系仅由计算图的原始张量流动与直接算子边提取; 若排除边界 COPY 收缩后引入了无法证明的附加依赖 (ambiguous 依赖), 或加入同核流水线次序弧后增广图出现环路, 分析器均主动抛出 **UnsupportedBound** 异常退出;
4. **合法性范围声明**: 该模块仅证明在给定次序下成功物理执行的完成时间下界, 并不构成该调度方案满足官方全部编译依赖或硬件执行合法性的证明。

当方案通过上述守卫后, 方案在任何一次成功的物理执行中, 各算子的启动时刻必定受到一组必要时序不等式的严格约束:

1. **数据依赖不等式**: 若算子 u 与算子 v 存在直接张量依赖或直接算子依赖, 则 v 的启动时刻满足:

$$s_v \geq s_u + d_u + \delta_{uv}. \quad (6.8)$$

其中, 若 u 与 v 被分配到不同的计算核心, 计入跨核心通信引发的硬件延迟 $\delta_{uv} = 500 \text{ cycles}$ (该数值对应冻结的 P3 官方配置); 若在同一核心, 则 $\delta_{uv} = 0$ 。同一对算子间若存在多个并列依赖约束, 系统取延迟的最大值 ($\max$), 不进行代数累加;

2. **同核同流水线 FIFO 约束**: 在同一计算核心内部, 分配给同一类执行流水线 (如 PIPE_M) 的各个算子必须按照调度方案排定的先后次序单串行推进执行。若在该核心的该流水线上算子 u 紧排在算子 v 之前, 则必有:

$$s_v \geq s_u + d_u. \quad (6.9)$$

依据上述必要不等式, 记 $ES(v)$ 为算子 v 的最早可能启动时刻。对于无前序约束的初始就绪算子, 令其候选基准为 0; 对于一般算子 v , 其最早启动时刻严格满足拓扑动态规划式:

$$ES(v) = \max(\{0\} \cup \{ES(u) + d_u + \delta_{uv} \mid (u, v) \in \mathcal{E}_{\text{bound}}\}). \quad (6.10)$$

整份方案的物理完成时间下界 $L(P)$ 定义为所有算子执行完成时刻的最大值 (空图时取 0):

$$L(P) = \begin{cases} \max_{v \in V} (ES(v) + d_v), & V \neq \emptyset, \\ 0, & V = \emptyset. \end{cases} \quad (6.11)$$

该下界完全由底层硬件执行的必要不等式推导得出。若当前系统已经取得了一份完成时间为 U 的合法方案, 且计算得出 $L(P) \geq U$, 则该候选方案在实际执行中的 **Makespan** 必然满足 $M \geq L(P) \geq U$, 绝不可能严格优于现有结果, 从而可以被安全剪除。需要指出的是, 该下界是必要约束界定的理论底线, 并不宣称其数值必然等于消除外部瓶颈后实际可达的调度耗时; 相较之下, 任意放松约束后运行贪心启发式算法所得的耗时仅代表某一种可行排程的完成时间, 并不构成理论下界。

6.3.4 在线择优与回退

图 6.3-1 完整呈现了问题三的树状排序准则与全流程候选方案的评测决策逻辑:

1. **树状排序的触发与适用限定**: 排序机制严格限定于满足特定结构条件的归约森林 (由 `forest_memory_order.py` 判定: 至少包含两个树状连通组件且存在多子算子汇聚结构; 每个算子均为已知尺寸的单一输出, 非根算子输出仅被唯一父算子消费, 根算子输出作

为外部依赖)。对满足条件的各子树，算法依据指标差值 $h_i - r_i$ （其中 h_i 为子树内部结果的存储峰值字节数， r_i 为子树完成后保留的输出字节数）按降序（即非升序）排定处理次序，并在同值时按算子编号升序破平，随后执行父算子。该准则在非交错树模型推导下保证单棵树内部结果的局部存储峰值达到最小；需要强调的是，该理论保证仅针对树模型局部峰值指标，并不改变既定的分核归属，亦不直接代表物理 L1 或片上 Unified Buffer 的真实占用峰值，不推断官方评测 Makespan 缩短或缓存命中率必然提升；

2. **完整候选的评测决策链：**在求解流程中，所有候选方案共享至多 3 次在线 E0 评测预算。基础方案与注意力感知候选先行生成并确立暂定最优解（当前最优方案 winner，指迄今已完成评测的候选方案中具有最小官方 Makespan 的方案，并非全局理论最优解）；针对新构造的树状排序方案，系统依次实施前置检验：若已用评测次数达到 3 次，或无法构造合法的树状排序方案，或与当前最优方案（winner[0]）直接比对编码字节完全相同，则直接跳过评测，避免重复评测（不维护历史全量去重表）；
3. **理论下界过滤与严格改善：**若该方案满足下界构建条件且计算所得下界 $L(\text{proposal}) \geq U$ ，则断定其完成时间绝不可能严格超越当前最优值，直接作为 bound_pruned 剪除；若由于拓扑或算子类型导致下界不支持（UnsupportedBound），算法记录 bound_unavailable 并继续推进评测，绝不将无法证明的下界当作剪枝条件；仅当通过全部前置检验时，方调用官方 E0 进行最终评测，且仅在 Makespan 严格缩短时采纳该方案。

算法 6.2 问题三归约森林与多候选协同求解算法

```

1 # 输入：拓扑索引 index，核心数 cores，评测器 evaluate，硬件跨核延迟
   delay
2 # 输出：选中的调度方案 winner[0]，评测次数 calls（全链共用 <= 3 次预算）
3 winner, calls, _, _ = witness_solve.evaluate_candidates(index, cores
   , evaluate, save)
4 if calls >= 3:
5     return winner[0], calls # 评测预算耗尽直接返回
6
7 # 构造树状排序方案（伪代码抽象：结构不支持安全返回 None）
8 proposal = safe_construct_forest(index, cores)
9 if proposal is None or encoded(proposal) == encoded(winner[0]):
10     return winner[0], calls # 结构不支持或与当前最优方案重复直接返回
11
12 # 依据硬件依赖与 FIFO 必要不等式求解下界（UnsupportedBound 不误剪）
13 try:
14     bound = pipe_bound.analyze(index.graph, proposal, delay)
15     lower = bound["with_cross_core_delay"]["lower_bound_cycles"]
16     if lower >= winner[1]["makespan"]:
17         return winner[0], calls # 下界不小于当前完成时间，无法严格改善
   则剪枝
18 except UnsupportedBound:
19     pass
20
21 # 在剩余预算内发起评测，仅当 Makespan 严格更小时采纳（平局不以 DDR 破
   平）
22 calls += 1
23 score = safe_evaluate(evaluate, proposal) # 仅捕获数据校验异常，返回
   None

```

```

24 if score and score["makespan"] < winner[1]["makespan"]:
25     winner = (proposal, score, "forest_memory_order")
26 return winner[0], calls

```

源码对应: 311322b, src/q3/forest_solve.py::evaluate_candidates (其中 safe_construct_forest 封装结构守卫, safe_evaluate 仅捕获 EvaluationValidationError 校验异常并回退 None, 不捕获其余异常)。全流程共用至多 3 次在线 E0 评测预算且在构造前先查预算; 下界依据硬件时序与 FIFO 必要不等式推导, 下界不小于当前完成时间时跳过评分 ($L \geq U$ 意味着已不能严格改善当前方案); 下界不适用则正常评测; 新方案仅在官方 Makespan 严格缩短时采纳, 平局不以 DDR 搬运量破平。

6.3.5 时间复杂度分析

算法在求解过程中的计算耗时由以下几个相互独立的计算环节组成:

1. **森林识别与拓扑分析:** 在无向连通分量与依赖拓扑提取中, 对计算算子及连通分量的遍历耗时为 $O(V + E)$;
2. **自底向上指标递推与子树排序:** 自叶向根递推计算各节点存储指标需 $O(V)$ 时间。在每个汇聚节点处对其子节点进行排序, 设节点 v 的入度 (即前序子树数量) 为 $\deg(v)$, 全图排序的总时间复杂度为:

$$O\left(\sum_{v \in V} \deg(v) \log \deg(v)\right) \leq O(V \log V). \quad (6.12)$$

3. **深度优先后序遍历与方案拼接:** 生成各分量 DFS 序列与拼接核心序列的基础遍历耗时为 $O(V)$; 对分配至各个核心的分量, 按照各分量子树最小算子编号进行确定性升序排序的开销另计为 $O\left(\sum_{k=0}^{K-1} C_k \log C_k\right) \leq O(C \log C)$ (其中 C_k 为分配至核心 k 的连通分量数, $\sum_{k=0}^{K-1} C_k = C \leq V$);
4. **必要时序图构建与最长路径求解:** 若候选方案满足单操作 M/V 算子条件, 从原图中提取直接算子依赖边并进行端点排序去重耗时 $O(E_o \log E_o)$ (E_o 为原图算子间依赖边数); 在构建好的包含 V 个节点与 E_{bound} 条约束弧的有向无环增广图上, 利用拓扑动态规划求解最长路径的时间复杂度为 $O(V + E_{\text{bound}})$;
5. **官方离散事件仿真评测开销:** 在全流程至多 $b \leq 3$ 次在线评测调用中, 仿真耗时表示为 $\sum_{j=1}^b T_j$, 其中 T_j 取决于官方离散事件仿真器的内部模拟事件量。

需要明确说明的是, 上述森林排序部分的复杂度界仅适用于该排序生成逻辑本身, 并不包含外部全局 Index 索引构建、初始分量核心指派 (assignment) 以及官方底层编译与仿真评测的全局耗时, 避免将整个求解流程错误归属为线性或低多项式复杂度。同时, 评测次数上限为常数 (≤ 3) 亦不构成全流程计算时间的代数证明, 求解耗时的可控性根本上依赖于前置结构守卫与下界剪枝的配合。

6.4 结果与分析

6.4.1 共享缓存的配对实验

本文算法 (Forest 固定版本, 源码对应 Commit 311322b) 在官方提供的 100 张全量基准计算图、涵盖 1 至 5 核心的全部 500 种测试组合上进行了完整的端到端求解与评测, 所有方案均顺利通过官方合法性校验。测试结果依据 paper/manuscript-v1/data/all-results.csv 与固定数据资产进行了严格复核。

在全量 500 个测试组合的求解过程中, 求解器累计发起 973 次在线 E0 评测, 未触发任何重试。与前一完整策略相比, 全网格共有 24 个用例实现了 Makespan 的进一步缩短,

其余 476 个用例持平，零退化（ $\text{degrade} = 0$ ）。在额外数据搬运量（ extra_ddr_bytes ）与由于片上容量不足引发的搬运量（ spill_bytes ）方面，各有 18 个用例减少，479 个用例持平，3 个用例增加，两项统计指标均实现了 413,450,240 字节的净减少。

为深入量化共享只读 Cache 对系统性能的实际影响，实验对同一套生成的 P3 最终方案分别在“关闭 Cache（无 L2）”与“开启 Cache”两种官方评测配置下进行了成对（paired）仿真对比。下表汇总了各核心数下的测试数据（注意：表中“无 L2”列代表同一份 P3 方案在关闭 Cache 时的表现，并非由问题二算法独立生成的方案）：

表 6.2 共享缓存性能收益配对实验

计算核心数 量	无 L2: $\text{mean}(B/M_2)$	开启 Cache: $\text{mean}(B/M_3)$	成对加速比均值 $\text{mean}(M_2/M_3)$	逐用例字节命中 率算术平均
1	1.1971	1.1998	1.0023	6.47%
2	2.2949	2.3120	1.0079	16.25%
3	3.1745	3.2587	1.0314	25.96%
4	3.9211	4.0915	1.0512	24.53%
5	4.4763	4.7576	1.0829	31.36%

表中数据及各核心数下的加速比表现（图 6.4-1）展现了如下规律：

- Cache 带来的增益表现：**在 1 至 5 核心下（Cache 字节命中率与成对性能增益的分布关系见图 6.4-2），开启 Cache 后方案的平均加速比从无 Cache 时的 1.1971、2.2949、3.1745、3.9211 与 4.4763，分别提升至 1.1998、2.3120、3.2587、4.0915 以及 4.7576。逐用例成对加速比均值 $\text{mean}(M_2/M_3)$ 在 5 核心场景下达到 1.0829；
- 指标口径的精确区分：**表中第一、二列的分子基准 B 均严格采用官方单核基准方案的 Makespan。第三列为逐用例计算 M_2/M_3 后的算术平均值（1.0829），该数值在数学上并不等于两列均值的直接相除（ $\frac{4.7576}{4.4763} \approx 1.0628$ ）；
- 缓存命中率的统计差异：**在 5 核心场景下，逐用例字节命中率的算术平均值为 31.36%。若将全量 100 张图的全部访问请求汇总后统一计算，按整体访问字节加权的总字节命中率为 47.45%。两者差异源于不同计算图的通信规模不均，访问数据量庞大的计算图在加权统计中占据了更高比重；
- 求解端到端耗时分布：**包含在线评测在内，求解程序在 500 个测试组合上的墙钟时间均值为 4.401 秒，中位数为 1.616 秒，95% 分位数（最近秩次法，第 475 位）为 20.009 秒，最大值为 51.335 秒。

图 6.4-3 进一步展示了问题三 Forest 主算法（311322b）在全量 500 个测试组合上的方案质量与求解耗时分布：

- 质量与耗时双对数分布：**子图 (a) 为官方 Makespan (cycles) 与完整求解墙钟时间（秒）的双对数散点分布，呈现了调度解质量与求解耗时的对应关系；
- 分核经验累积分布：**子图 (b) 展示了各核心数（每组 100 用例）求解耗时的经验累积分布函数（ECDF）。在分核统计中，各核心数下的求解耗时中位数介于 0.989 秒至 2.176 秒之间；各核心数的 95% 分位数（线性插值）中，最大分核 P95 为 20.887373 秒（发生于 5 核心场景），需要明确指出该数值为分核心子集的最大 P95，而非全量 500 个测试组合的整体 P95；全量 500 个测试组合的单次最大求解耗时为 51.335296 秒；
- 测量环境与口径说明：**测试环境为配备 Apple M5 Pro 芯片的主机（48 GiB 内存，macOS 27，Python 3.12.13，每分片 worker = 1，全局至多 4 个分片并发，非独占机器），求解墙

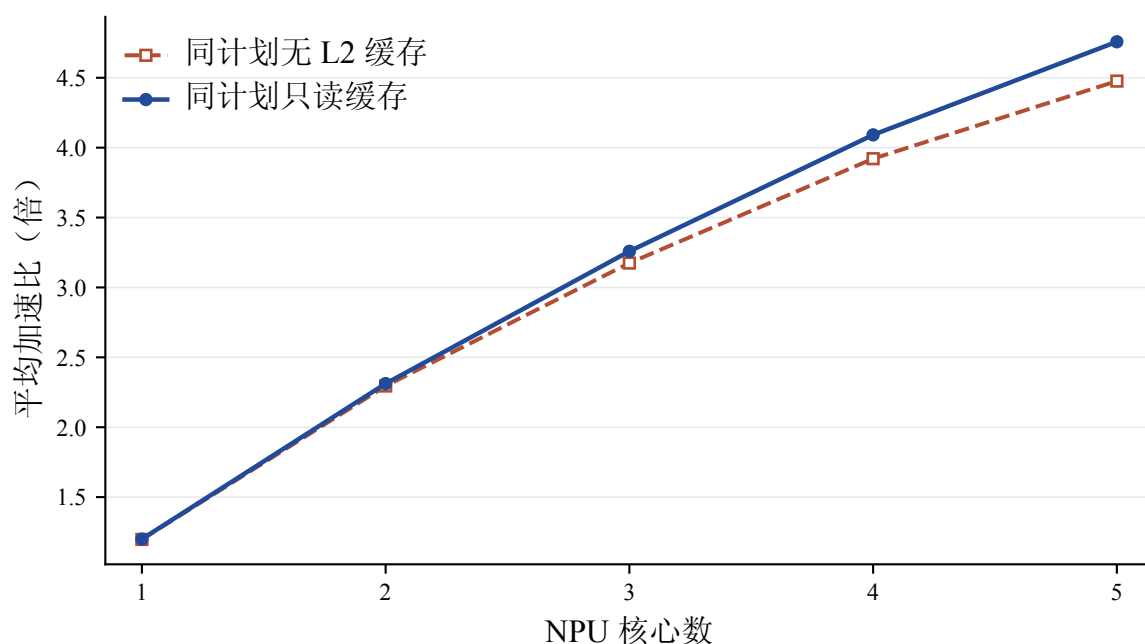


图 6.4-1 问题三开启只读 Cache 与同方案无 L2 运行的平均加速比对比。展示 1 至 5 核心下（每组 100 对同方案）逐用例计算 B/M 后的算术平均值；单核保留实际比值（无 L2 1.197143，只读 Cache 1.199760），不强制设定为 1。全量 500 对数据源于同计划机制对比，不替代独立优化的问题二主算法方案。

钟包含读图、方案构造、在线 E0 评测与文件持久化，无独立的外部最终评测耗时记录（对应空值，不可视作 0 耗时）。

6.4.2 启发式选择条件的消融分析

为了探索能否在在线评测阶段增加多重评价指标以进一步优化 Cache 利用效果，前期研发中曾设计了一组附加筛选条件的对照实验（代码代号为 R9F）。

在 R9F 策略中，设计者对 5 核心的候选方案设置了三项必须同时满足的严格筛选准则：

1. 开启 Cache 后的完成时间严格缩短 ($M_{3,new} < M_{3,old}$)；
2. 关闭 Cache 后的完成时间不退化 ($M_{2,new} \leq M_{2,old}$)；
3. Cache 带来的相对改善比值不下降，即 $G_{new} \geq G_{old}$ ，其中比值定义为 $G = \frac{M_2}{M_3}$ 。

在 5 核心的 100 张基准计算图中，96 张图由于不符合该策略的特定前置结构而跳过，1 张图被下界分析模块提前剪枝，其余 3 张图生成了候选方案并进入了在线评测。然而，这 3 张图生成的候选方案均因未能同时满足上述三项准则而被判定拒绝，最终 R9F 输出的 500 份方案与 Forest 主算法完全一致。

对这 3 张被拒绝用例的深层数据复核揭示了一个关键反例现象：这三份被拒绝的候选方案在实际执行中，其实际执行时间 M_2 与 M_3 均有缩短，具体数值如下表所示：

因为执行完成时间恒为正整数，准则 3 中要求的比值不下降条件可严格展开为代数乘积形式：

$$G_{new} \geq G_{old} \iff \frac{M_{2,new}}{M_{3,new}} \geq \frac{M_{2,old}}{M_{3,old}} \iff M_{2,new} \cdot M_{3,old} \geq M_{2,old} \cdot M_{3,new}. \quad (6.13)$$

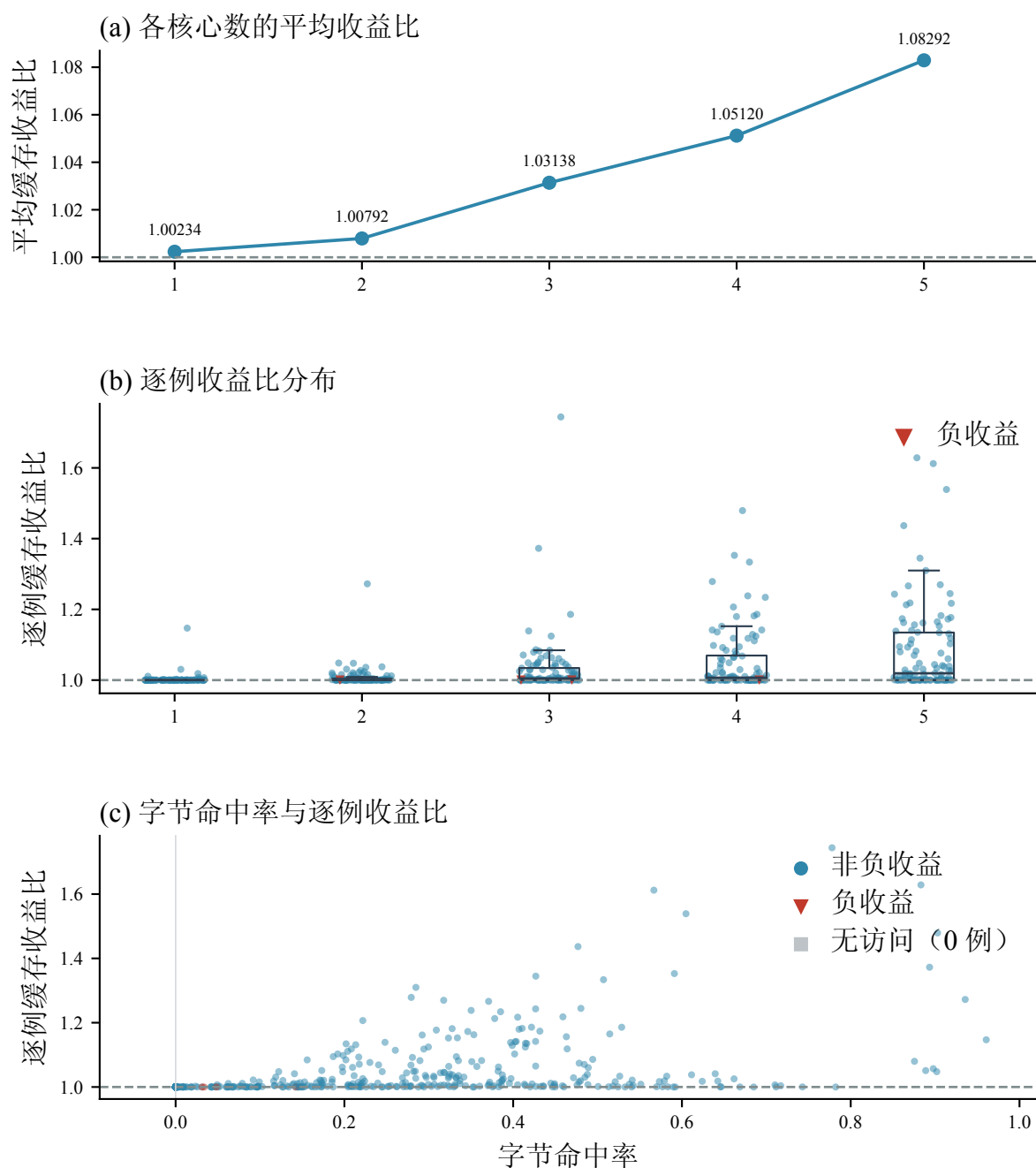


图 6.4-2 问题三只读 Cache 性能增益 (CacheGain) 与字节命中率分布。面板 1 展示 1 至 5 核心下 500 对同方案的平均成对加速比 (均值由 1.0023 提升至 1.0829); 面板 2 展示各核心数的逐例分布与四分位箱线 (包含 CacheGain < 1 的负收益样本, 以红色三角标注); 面板 3 呈现字节命中率与 CacheGain 的逐例散点分布 (点集仅客观反映分布特征, 不构成命中率单独决定加速比的证据)。

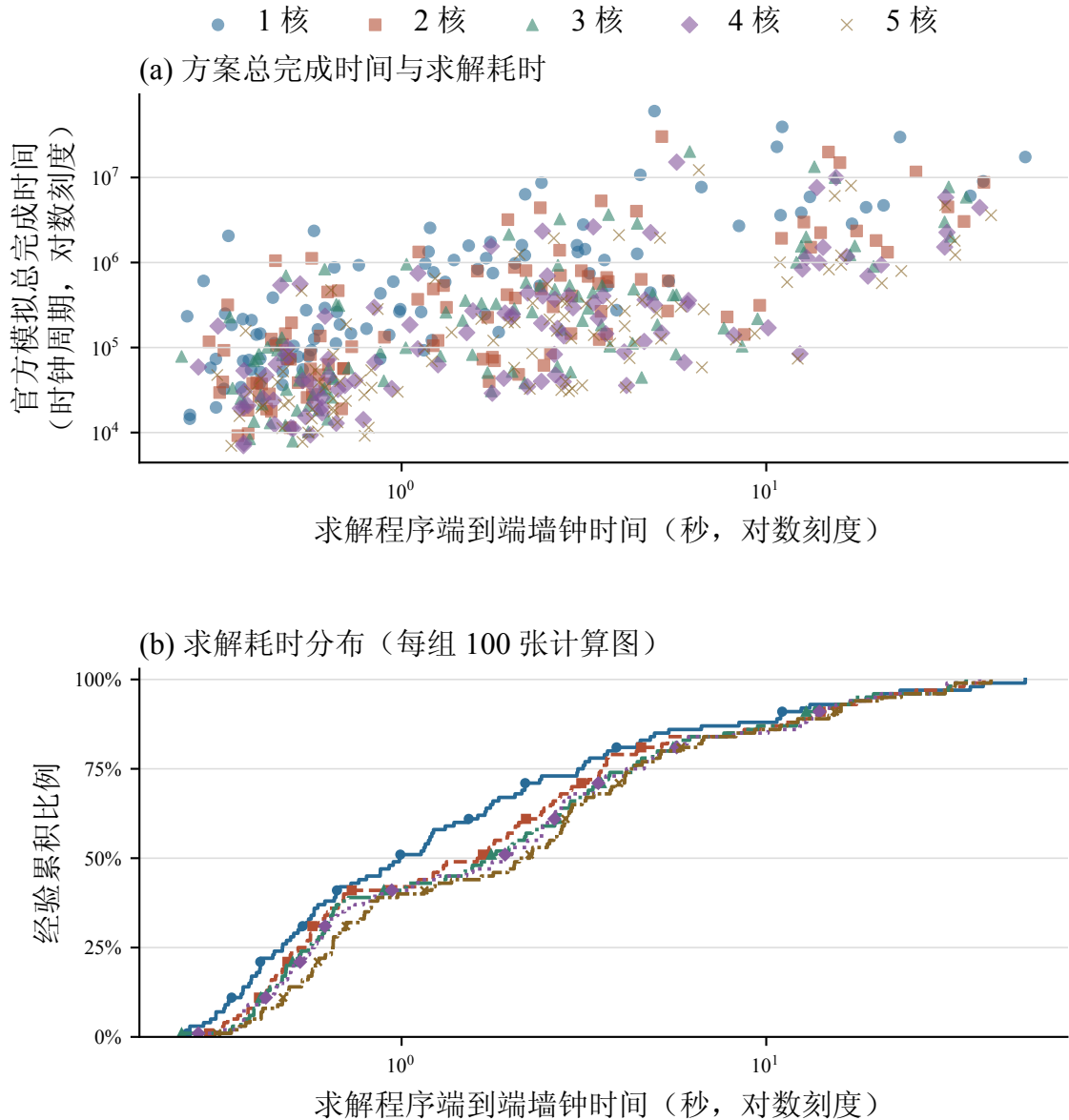


图 6.4-3 问题三 Forest 算法方案质量与求解耗时分布。上图展示 500 个测试组合在官方离散事件仿真下的 Makespan 与求解器端到端墙钟时间双对数散点分布；下图展示各核心数下（每组 100 用例）求解耗时的经验累积分布函数（ECDF）。评测在配备 Apple M5 Pro 芯片的主机（48 GiB 内存，macOS 27，Python 3.12.13，每分片 worker = 1，全局至多 4 分片并发）环境下完成，计时涵盖子进程启动、图解析、在线 E0 选优与方案写出全流程。

以 case_088 为例，原方案的指标为 $M_2 = 97,305$ cycles, $M_3 = 85,789$ cycles, 比值 $G_{\text{old}} \approx 1.1342$ 。候选方案将无 Cache 时的完成时间大幅压缩至 $M_{2,\text{new}} = 66,729$ cycles (缩短了 31.42%)，同时将开启 Cache 时的完成时间压缩至 $M_{3,\text{new}} = 66,224$ cycles (缩短了 22.81%)。由于无 Cache 场景下的优化幅度超越了有 Cache 场景下的优化幅度，导致新方案的比值下降为 $G_{\text{new}} \approx 1.0076 < G_{\text{old}}$ 。该比值准则因此直接判定该候选不合格并予以抛弃。

表 6.3 启发式选择条件消融与反例分析 (R9F)

测试用例 / 核心数	原方案 M_2 / cycles	候选方案 M_2 / cycles	原方案 M_3 / cycles	候选方案 M_3 / cycles
case_064 / 5 Core	7,029	6,899	6,981	6,877
case_068 / 5 Core	131,631	98,913	116,345	97,971
case_088 / 5 Core	97,305	66,729	85,789	66,224

这一消融实验清楚地证明：在多目标多场景评估中，人为构造的复合比值指标容易产生与核心目标背离的掩盖效应，反而排除了在官方模拟中能够大幅缩短执行时间的优秀方案。本文主算法 Forest 坚持以官方规定的主口径 Makespan 严格单调下降作为唯一的接受准则，确保了优化方向始终紧密对准赛题的首要目标。

6.5 本章小结

本章针对共享只读 Cache 与 FIFO 逐出语义，建立非交错树状模型，严格按子树存储峰值与保留输出之差的降序安排执行次序，辅以关键路径下界剪枝与在线仿真选优。在 5 核心场景下平均加速比达到 4.7576；同一方案开关 Cache 配对仿真的平均成对增益为 1.0829，平均字节命中率 31.36%。排序结论严格受限于非交错树结构，且缓存命中并不保证整体完成时间缩短（如已测旧固定方案反例增加 143 周期）。

7 综合实验与结果分析

7.1 实验设置

本文算法在官方赛题指定的全部 100 张原始计算图上展开系统性实验，计算核心数量依次配置为 1 至 5 核心，每道题目涵盖 500 种“计算图—核心数”测试组合，三道题目合计完成 1500 组完整的端到端求解与仿真评测。针对每一道题目，实验均采用运行前已完全冻结的单一固定求解器程序版本，其内部策略选择规则、候选生成数量以及在线评测调用上限在运行前全部固化，不针对特定算题进行人工微调。

实验数据由统一的数据资产库（`paper/manuscript-v1/data/all-results.csv` 与 `source-receipt.json`）导出并复核。全文分析依托于三类明确的对比维度：

- 相对官方单核心基准的加速比：**以官方提供的单核心基准方案完成时间除以算法输出方案的完成时间，严格计算每一测试用例的加速比，并计算算术平均值；
- 算法版本演进的宏观对比：**比较同一计算图和同一核心数下，不同完整算法版本之间的整体性能演进。由于每个主版本通常包含结构识别、核心放置与顺序调整等多项改进，宏观版本对比用于展示系统综合能力的提升，不将整体增益简单归因于单一机制的孤立贡献；
- 共享只读 Cache 的成对隔离对比：**针对问题三生成的同一份最终调度方案，保持任务切分与排程完全不变，分别在关闭 Cache（无 L2）与开启 Cache 两种官方硬件配置下运行离散事件仿真，严格度量共享 Cache 对同一物理调度方案执行时钟周期的纯收益。

在实验数据整理、统计口径核对与结果分析过程中，本研究采用了人工智能工具提供辅助支持。所使用的工具为 GPT-6 Astra（API 型号：gpt-6-astra，由 OpenAI 开发，版本发布日期为 2026 年 9 月 3 日）[20]。文中全部测试用例的实测数值均严格源自前述已冻